

Python Interview Questions — 5+ Years

1. Core Python

1. What are the differences between **list, tuple, set, and dictionary**?
2. What is the difference between **mutable and immutable** objects?
3. Explain **shallow copy vs deep copy**.
4. What is the difference between **is** and **==**?
5. Explain Python's **memory management**.
6. What are ***args** and ****kwargs**?
7. What are **lambda functions**?
8. What is list comprehension? When should you avoid it?
9. What are generators? Why use **yield**?
10. Difference between an **iterator and iterable**.
11. What are decorators? Give a real-world use case.
12. Explain Python **context managers** and the **with** statement.
13. What is exception handling? Explain **try, except, else, and finally**.
14. What is the difference between **pass, continue, and break**?
15. What are Python namespaces and scopes?
16. Explain **LEGB rule**.
17. What is duck typing?
18. What is monkey patching?
19. What is **__init__()** vs **__new__()**?
20. What are magic/dunder methods?

2. Python Coding Questions

21. Reverse a string without using **[::-1]**.
22. Find duplicate elements in a list.
23. Find the **second-highest number** in a list.
24. Count the frequency of each character in a string.

25. Find the first non-repeating character.
26. Check whether two strings are anagrams.
27. Remove duplicates while preserving order.
28. Find common elements between two lists.
29. Flatten a nested list.

Example:

```
input = [[1, 2], [3, 4], [5, 6]]  
  
output = [1, 2, 3, 4, 5, 6]
```

LinkedIn@AshishZope

30. Find missing numbers from a sequence.

3. Advanced Python

31. What is the **GIL** in Python?
32. Multithreading vs multiprocessing — when would you use each?
33. What is **asyncio**?
34. Difference between **process, thread, and coroutine**.
35. How would you process a **10 GB CSV file** without loading it completely into memory?
36. How can you optimize a slow Python program?
37. What is profiling? Which tools have you used?
38. What is garbage collection in Python?
39. How does Python dictionary lookup work?
40. What is the average time complexity of:
 - List lookup
 - Dictionary lookup
 - Set lookup
41. How does Python handle memory allocation?
42. What are closures?
43. Explain function decorators with an example.

- 44. What is serialization/deserialization?
 - 45. Difference between **pickle and JSON**.
-

4. Python + Data Engineering ★

- 46. How would you read a **large CSV file efficiently**?
 - 47. How would you process millions of records using Python?
 - 48. Pandas vs PySpark — when would you use each?
 - 49. How do you handle **missing/null values**?
 - 50. How do you remove duplicates from a large dataset?
 - 51. How would you perform incremental processing using Python?
 - 52. How would you implement a **retry mechanism** for an API call?
 - 53. How do you handle API failures and timeouts?
 - 54. How would you implement logging in a production Python application?
 - 55. How would you read data from PostgreSQL using Python?
 - 56. How would you efficiently insert millions of records into PostgreSQL?
 - 57. How do you prevent SQL injection in Python?
 - 58. How would you design a Python ETL pipeline?
 - 59. How would you make an ETL process **idempotent**?
 - 60. How would you implement **error handling and dead-letter processing**?
-

5. Python + PySpark 🔥

- 61. What is the difference between **Python list and Spark DataFrame**?
- 62. Why is PySpark slower than Scala Spark in some scenarios?
- 63. What is a **UDF** in PySpark?
- 64. Why should you avoid Python UDFs when possible?
- 65. What is a Pandas UDF?
- 66. Explain Spark **lazy evaluation**.
- 67. What is a Spark transformation vs action?

- 68. Explain `map()`, `mapPartitions()`, and `flatMap()`.
 - 69. What is the difference between `repartition()` and `coalesce()`?
 - 70. How do you handle data skew?
 - 71. How does broadcasting work in Spark?
 - 72. What is the difference between **cache** and **persist**?
 - 73. How would you optimize a slow PySpark job?
 - 74. How do you process a 1 TB dataset using PySpark?
 - 75. Explain Spark's execution model from **job** → **stage** → **task**.
-

6. Real-World Scenario Questions ★ ★ ★

These are especially important for your experience level:

- 76.** Your Python ETL job normally takes 20 minutes but suddenly takes 2 hours. How would you troubleshoot it?
- 77.** You receive a **100 GB CSV file** every day. Design a Python-based ingestion process.
- 78.** An API allows only **100 requests/minute**, but you need to process 10 million records. How would you design it?
- 79.** Your pipeline fails after processing 70% of the records. How would you restart it without duplicating data?
- 80.** PostgreSQL contains 500 million records. You need to extract only today's changed records using Python. How would you design it?
- 81.** Your Python process is consuming 20 GB RAM. How would you identify and fix the problem?
- 82.** You need to process files arriving continuously in cloud storage. How would you design the Python pipeline?
- 83.** A downstream API randomly returns **500**, **502**, and **503** errors. How would you implement retries?
- 84.** Your Python code works locally but fails in production. What would you check?

85. You have duplicate records arriving from multiple sources. Design a Python-based deduplication strategy.

7. Most Important for Your Interview

For a **Senior Data Engineer**, I would prioritize these **15 questions**:

Priority	Topic
★ ★ ★	Generators & <code>yield</code>
★ ★ ★	Decorators
★ ★ ★	GIL
★ ★ ★	Multithreading vs multiprocessing
★ ★ ★	Memory optimization
★ ★ ★	Large-file processing
★ ★ ★	Exception handling & retries
★ ★ ★	Logging
★ ★ ★	Pandas vs PySpark
★ ★ ★	Python UDF vs Pandas UDF
★ ★ ★	API ingestion
★ ★ ★	Incremental ETL
★ ★ ★	Idempotency
★ ★ ★	PostgreSQL + Python
★ ★ ★	Python + PySpark optimization

If you're preparing for a **Senior Data Engineer interview**, I can also give you a **100-question Python interview set with answers, coding problems, and real-world scenarios**, arranged from **Basic → Intermediate → Advanced → Data Engineering → PySpark**.